

Commandes Git Utiles

Voici un guide des commandes Git les plus courantes pour gérer vos dépôts locaux et distants (GitHub, GitLab), avec description et exemples.

Tableau de référence des commandes indispensables Git

Commande	Usage	Description	Exemple
git add	- git add [nom_fichier] - git add .	- Ajoute un fichier à l'index (prépare pour le commit) - Ajoute tous les fichiers dans le dossier courant à l'index	- git add index.html -git add .
git commit	git commit -m "message"	Crée un commit avec un message décrivant les modifications	git commit -m "Correction bug login"
git push	git push	Envoie vos commits locaux vers le dépôt distant	git push

Tableau de référence des commandes principales Git

Commande	Usage	Description	Exemple
git clone	git clone [URL]	Clone un dépôt distant sur votre machine et configure automatiquement le remote origin	git clone https://gitlab.com/user/projet.git

Commande	Usage	Description	Exemple
<code>git fetch</code>	<code>- git fetch</code> <code>- git fetch origin</code>	- Récupère les branches et commits distants sans fusionner et met à jour les références (pointeurs vers les commits montrant les différences entre remote et local) - Récupère uniquement le remote origin (commandes souvent équivalentes car souvent un seul remote, qui est origin)	<code>- git fetch - git fetch origin</code>
<code>git status</code>	<code>git status</code>	Affiche l'état des fichiers (modifiés, ajoutés, non suivis) et indique si la branche locale est en avance ou en retard par rapport à sa branche distante	<code>git status</code>
<code>git pull</code>	<code>- git pull - git pull origin [nom_branche]</code>	- Récupère et fusionne automatiquement les commits depuis la branche remote du dépôt distant dans la branche courante - Fait fetch origin puis merge avec la branche remote appelée (commandes souvent équivalentes)	<code>- git pull - git pull origin main</code>
<code>git init</code>	<code>git init</code>	Initialise un nouveau dépôt Git local dans le dossier courant	<code>git init</code>

Commande	Usage	Description	Exemple
<code>git remote add origin</code>	<code>git remote add origin [URL]</code>	Ajoute un dépôt distant nommé "origin" à votre dépôt local (cela ne rend pas votre dépôt local accessible sur Github/GitLab, mais configure seulement un lien vers un dépôt distant existant déjà créé sur Github/GitLab)	<code>git remote add origin https://gitlab.com/user/projet.git</code>
<code>git checkout -b</code>	<code>- git checkout -b [nom_branche]</code> <code>- git checkout -b [nom_branche] origin/[nom_branche]</code>	- Crée une nouvelle branche et s'y positionne - Après un fetch, crée une branche locale à partir d'une branche distante et s'y positionne	<code>- git checkout -b feature-login - git checkout -b feature-login origin/feature-login</code>
<code>git checkout</code>	<code>git checkout [nom_branche]</code>	Change de branche	<code>git checkout develop</code>
<code>git branch -m</code>	<code>git branch -m [ancien_nom] [nouveau_nom]</code>	Renomme une branche locale	<code>git branch -m develop dev</code>

Commande	Usage	Description	Exemple
<code>git push origin</code>	<code>- git push</code> <code>-u origin</code> <code>[nom_branche]</code> <code>- git push</code> <code>origin</code> <code>[nom_branche]</code>	- Pousse (envoie) les commits locaux vers le dépôt distant et définit la branche upstream (la branche remote associée) pour les futurs push/pull - Pousse les commits de la branche locale vers la branche remote spécifiée (même si ce n'est pas son upstream)	<code>- git push -u origin</code> <code>feature-login -</code> <code>git push</code> <code>origin dev</code>
<code>git merge</code>	<code>- git merge</code> <code>[nom_branche]</code> <code>git merge</code> <code>origin/</code> <code>[nom_branche]</code>	- Fusionne une branche dans la branche courante - Fusionne une branche remote dans la branche courante	<code>- git merge</code> <code>feature-login -</code> <code>git merge</code> <code>origin/feature-signup</code>
<code>git branch</code>	<code>git branch</code>	Liste les branches locales	<code>git branch</code>
<code>git log</code>	<code>- git log -</code> <code>git log</code> <code>--all</code>	- Affiche l'historique des commits de la branche courante - Affiche l'historique des commits de toutes les branches	<code>- git log - git</code> <code>log --all</code>
<code>git branch -r</code>	<code>git branch -r</code>	Liste toutes les branches distantes que le dépôt local connaît	<code>git branch -r</code>
<code>git branch -d</code>	<code>git branch</code> <code>-d</code> <code>[nom_branche]</code>	Supprime une branche locale (uniquement si elle a été entièrement mergée dans la branche courante ou dans sa branche remote)	<code>git branch -d</code> <code>feature-login</code>

Commande	Usage	Description	Exemple
<code>git branch -vv</code>	<code>git branch -vv</code>	Permet de voir l'upstream (la branche distante associée à la branche locale) : liste toutes les branches locales, leurs branches remotes associées, leur statut ahead/behind et leur dernier commit	<code>git branch -vv</code>

Tableau de référence des commandes secondaires Git

Commande	Usage	Description	Exemple
<code>git diff</code>	<code>git diff [nom_fichier]</code>	Montre les différences entre fichiers modifiés et dernier commit	<code>git diff fichier.txt</code>
<code>git diff</code>	<code>git diff [nom_branche1] [nom_branche2]</code>	Montre les différences entre deux branches	<code>git diff main develop</code>
<code>git switch</code>	<code>git switch [nom_branche]</code>	Change de branche (alternative à checkout)	<code>git switch develop</code>
<code>git switch -c (local)</code>	<code>git switch -c [nom_branche]</code>	Crée une nouvelle branche et s'y positionne	<code>git switch -c feature-signup</code>
<code>git switch -c (remote)</code>	<code>git switch -c [nom_branche] origin/[nom_branche]</code>	Après un fetch, crée une branche locale à partir d'une branche distante et s'y positionne	<code>git switch -c feature-login origin/feature-login</code>
<code>git branch -D ou --delete</code>	<code>git branch -D [nom_branche]</code>	Supprime une branche locale même si non fusionnée	<code>git branch -D feature-login</code>

Commande	Usage	Description	Exemple
<code>git push origin --delete</code>	<code>git push origin --delete [nom_branche]</code>	Supprime une branche sur le dépôt distant (GitHub ou GitLab). Ne supprime pas la branche locale	<code>git push origin --delete feature-login</code>
<code>git remote -v</code>	<code>git remote -v</code>	Affiche les dépôts distants configurés (GitHub, GitLab, etc.)	<code>git remote -v</code>
<code>git stash</code>	<code>git stash</code>	Sauvegarde temporairement les modifications non committées pour nettoyer le workspace	<code>git stashgit</code> <code>stash pop</code> (pour récupérer les modifications) <code>git stash list</code> (pour lister les modifications)

Ordre classique

Code récupéré sur Gitlab :

- Sur Gitlab, dans le projet : cliquer sur “Code” (en bleu à droite), puis sur l’image à droite de “Clone with HTTP” (“Copy URL”).
- Dans le terminal, se placer là où on souhaite déposer son projet, puis taper la commande `git clone <url-du-repo>` (ne pas oublier de rajouter un `-s` après `http`).
- Rentrer son username et son mot de passe Gitlab.
- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push` si on est sur la branche master en local.
- Pour travailler sur une autre branche pour une autre fonctionnalité : `git checkout -b <nouvelle-branche>` ou `git branch <nouvelle-branche>` puis `git checkout <nouvelle-branche>` pour se positionner dessus.
- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push -u origin <nom-branche>` si on a créé une nouvelle branche en locale et qu’on souhaite la créer aussi sur Gitlab (utiliser le même nom de branche en local et sur Gitlab).

- Merge Request sur Github/Gitlab avec merge entre branches (d'une branche source vers une branche cible), et si besoin plus tard : `git pull` en local.

Projet créé en local puis poussé sur Gitlab :

Sur Gitlab :

- Cliquer sur "New Project" (en bleu à droite).
- Cliquer sur "Create blank project".
- Remplir "Project name".
- Dans "Project URL", choisir son nom ou son groupe.
- Laisser en "Private".
- Décocher "Initialize repository with a README".
- Cliquer sur "Create project".
- Une fois le projet créé, récupérer l'URL du projet en copiant ce qu'il y a après "git clone" (dans "Create a repository").

En local dans le terminal :

- Taper la commande : `git init`, puis la commande `git remote add origin <url-du-repo>` (ne pas oublier de rajouter un `-s` après `http`).
- Taper ensuite les commandes `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push -u origin <nom-branche>` (si vous êtes sur la branche master en local, il est recommandé d'appeler votre branche distante master aussi pour ne pas se perdre).
- Pour travailler sur une autre branche pour une autre fonctionnalité : `git checkout -b <nouvelle-branche>` ou `git branch <nouvelle-branche>` puis `git checkout <nouvelle-branche>` pour se positionner dessus.
- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push -u origin <nom-branche>` si on a créé une nouvelle branche en locale et qu'on souhaite la créer aussi sur Gitlab (utiliser le même nom de branche en local et sur Gitlab).
- Merge Request sur Github/Gitlab avec merge entre branches (d'une branche source vers une branche cible), et si besoin plus tard : `git pull` en local.

Notes rapides

Ajouter tous les fichiers modifiés avant un commit :

```
git add .
git commit -m "Message du commit"
git push
```

git add .

On sélectionne les fichiers qu'on veut "préparer" pour le commit.

Le point signifie "tous les fichiers modifiés dans le dossier courant".

En local seulement : ça touche uniquement notre dépôt sur notre ordinateur.

git commit -m "Message du commit"

On crée un snapshot (une copie instantanée, une photo de notre code à ce moment).

Le message aide à savoir ce que contient ce commit.

En local seulement : notre dépôt distant n'est pas encore touché.

git push (ou git push -u origin nom_branche)

Ici, on envoie nos commits depuis notre dépôt local vers le dépôt distant (GitHub/GitLab).

-u origin : c'est le nom par défaut de notre dépôt distant, configuré quand on a cloné notre projet.

nom_de_la_branche : c'est la branche sur laquelle on veut envoyer nos commits (souvent main ou master, ou une branche spécifique pour chaque fonctionnalité sur laquelle on travaille).

Si on est en local sur une branche appelée "feature-login", et qu'une branche distante sur Github/Gitlab existe aussi, faire **git push -u origin feature-login** va envoyer nos commits directement sur cette branche. Si cette branche n'existe pas sur Github/Gitlab, la commande **git push -u origin feature-login** va créer automatiquement une nouvelle branche distante appelée feature-login, et les commits locaux qu'on a fait seront envoyés vers cette nouvelle branche distante.